SANDIP FOUNDATION'S
SANDIP INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE
(An Autonomous Institute Permanently Affiliated to Savitribai Phule Pune University, Pune)
MAHIRAVANI, TRIMBAK ROAD, TAL & DIST: NASHIK-422213, MAHARASHTRA, INDIA

Practical No. 9

Title:

Simple Fractals Representation

Aim

To generate and display simple fractal patterns using recursive computer graphics techniques in C++ and OpenGL.

Objectives

- To understand the concept of fractals and self-similarity.
- To implement recursion for generating a simple fractal pattern.
- To understand how a complex graphical pattern can be generated from a simple repeated rule.
- To display the generated fractal using OpenGL.

Theory

A fractal is a geometric pattern that shows self-similarity, meaning that smaller portions of the pattern resemble the whole pattern. Fractals are generally generated by applying a simple rule repeatedly. Recursion is therefore an important technique used in fractal generation.

In computer graphics, fractals can be used to represent natural or mathematical patterns such as trees, mountains, coastlines and branching structures. At every recursive level, the current line or shape is divided or transformed into smaller parts. The recursion stops when a specified depth or minimum size is reached.

A simple recursive line fractal can be generated by dividing a line into smaller segments and applying the same procedure to each segment. The number of recursive levels controls the complexity of the final pattern.

Basic Fractal Concept

- Start with a basic geometric element.
- Apply a fixed transformation or subdivision rule.
- Repeat the rule recursively for a selected depth.
- Stop when the maximum recursion depth is reached.

Algorithm

1. Start the program.
2. Initialize OpenGL/FreeGLUT and create the graphics window.
3. Define the initial line segment or basic geometric shape.
4. Read the required recursion depth from the user.
5. If the recursion depth is zero, draw the current segment and return.
6. Otherwise, divide or transform the current segment according to the selected fractal rule.
7. Recursively apply the same rule to each smaller segment.
8. Continue until the required recursion depth is reached.
9. Display the generated fractal pattern in the graphics window.
10. Stop the program.

Working Principle

The important feature of the program is recursion. The same drawing function calls itself with smaller segments or a reduced recursion depth. Thus, a small number of instructions can generate a detailed graphical pattern.

Result

The program successfully generates and displays a simple fractal pattern using recursive subdivision in C++ and OpenGL. The appearance and complexity of the fractal change with the selected recursion depth.

Conclusion

Thus, the concept of simple fractal representation was successfully studied and implemented using recursion. The experiment demonstrates how repeated application of a simple rule can produce a complex graphical pattern.

SANDIP FOUNDATION'S
SANDIP INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE
(An Autonomous Institute Permanently Affiliated to Savitribai Phule Pune University, Pune)
MAHIRAVANI, TRIMBAK ROAD, TAL & DIST: NASHIK-422213, MAHARASHTRA, INDIA

Practical No. 10

Title:

Generate Fractal Patterns using (i) Bezier (ii) Koch Curve

Aim

To generate and display graphical patterns using Bezier curves and the Koch curve with recursive subdivision in C++ and OpenGL.

Objectives

- To understand the basic concept of Bezier curves and control points.
- To generate a smooth Bezier curve from a set of control points.
- To understand recursive construction of the Koch curve.
- To compare curve generation using parametric evaluation and recursive subdivision.
- To display the generated patterns using OpenGL.

Theory

Bezier Curve

A Bezier curve is a parametric curve defined by a set of control points. The curve starts at the first control point and ends at the last control point, while the intermediate control points influence its shape. For a cubic Bezier curve with control points P0, P1, P2 and P3, the curve is represented by:

$$B(t) = (1-t)^3P0 + 3(1-t)^2tP1 + 3(1-t)t^2P2 + t^3P3,\ 0 \le t \le 1$$

By evaluating t from 0 to 1 and joining the calculated points, a smooth Bezier curve can be displayed.

Koch Curve

The Koch curve is a well-known fractal generated by repeatedly replacing each line segment with four smaller segments. The middle portion of the original segment is replaced by two segments that form an outward triangular peak. Repeating this process increases the detail of the curve.

At recursion level 0, the curve is a straight line. At every higher level, each line segment is divided into three equal parts and the middle part is replaced by two segments forming an equilateral-triangle-like bump.

Algorithm — Bezier Curve

1. Start the program and initialize OpenGL/FreeGLUT.
2. Read or define four control points P0, P1, P2 and P3.
3. Set t = 0 and choose a small increment Δt.
4. Calculate the Bezier point using the cubic Bezier equation.
5. Increase t by Δt and calculate the next point.
6. Join successive points to form the curve.
7. Continue until t reaches 1.
8. Display the Bezier curve in the graphics window.

Algorithm — Koch Curve

1. Start the program and initialize OpenGL/FreeGLUT.
2. Define the starting and ending points of the initial line segment.
3. Read the required recursion depth.
4. If the recursion depth is zero, draw the current line segment.
5. Otherwise divide the segment into three equal parts.
6. Replace the middle third by two segments forming an equilateral triangular peak.
7. Recursively apply the same procedure to all four resulting segments.
8. Continue until the selected recursion depth is reached.
9. Display the complete Koch curve.
10. Stop.

Comparison

- Bezier curve: controlled by a finite set of control points and generated using a parametric equation.
- Koch curve: generated recursively by repeatedly replacing each segment with four smaller segments.
- Bezier curves are commonly used for smooth shapes, while the Koch curve demonstrates self-similar fractal geometry.

Result

The program successfully generates a smooth Bezier curve from control points and a Koch curve using recursive subdivision. The resulting graphical patterns are displayed using C++ and OpenGL.

Conclusion

Thus, Bezier curve generation and Koch curve fractal generation were successfully studied and implemented. The experiment demonstrates both parametric curve generation and recursive fractal construction in computer graphics.